

《数据库系统》期末复习知识点总结（全）

原创 乔卿 于 2020-12-15 21:05:54 发布 阅读量6.8w 收藏 3.6k 点赞数 481

分类专栏: 数据库系统 文章标签: 数据库 mysql



数据库系统 专栏收录该内容

71 订阅 19 篇文章

摘要 本文详细介绍了数据库系统的基础知识，包括数据库系统概述、概念模型和结构。重点讲解了关系数据库，涵盖关系数据结构、完整性约束和SQL语言。此外，还讨论了数据库安全性控制、数据恢复技术、并发控制以及NoSQL和New数据库的特点。内容深入，适合数据库初学者和进阶者。摘要由CSDN通过智能技术生成

目录

- 1. 数据库系统基础
 - 1.1 数据库系统概述
 - 1.1.1 基本概念
 - 1.1.2 数据独立性
 - 1.2 概念模型
 - 1.3 数据库系统的结构
- 2. 关系数据库
 - 2.1 关系数据结构及形式化定义
 - 2.2 关系的完整性
 - 2.2.1 实体完整性
 - 2.2.2 参照(引用)完整性
 - 2.3 关系代数
 - 1.选择(selection)
 - 2.投影(projection)
 - 3.连接(笛卡尔积)
 - 4.并UNION
 - 5.差minus (except)
 - 6.重命名 as
- 3. 关系数据标准语言SQL
 - 3.1 数据定义
 - 3.1.1 模式
 - 3.1.2 表
 - 3.1.3 索引
 - 3.2 数据查询
 - 3.3 数据更新
 - 3.3.1 插入元组
 - 3.3.2 插入子查询结果
 - 3.4 修改数据
 - 3.5 删除数据
 - 3.6 视图
 - 3.6.1 定义视图
 - 3.6.2 删除视图
 - 3.7 索引
 - 3.8 过程与函数
 - 3.8.1 过程
 - 3.8.2 函数
 - 4.1 数据库安全性控制
 - 4.2 对数据库不安全的因素
 - 4.3 数据库安全性控制
 - 4.4. 审计
 - 4.4.1 授权：授予与收回
 - 4.4.2 数据库角色（按岗位职责，组



乔卿

已关注

481



3677

13

分享

打赏



专栏

- 6. 关系数据理论
 - 6.1 规范化
 - 6.1.1 函数依赖
 - 6.1.2 第二范式(2NF)
 - 6.1.3 第三范式(3NF)
 - 6.1.4 BCNF(Boyce Codd Normal Form)
- 7. 数据库设计
 - 7.1. 数据库设计概述
 - 7.2. 数据库设计的基本步骤
 - 7.3. 数据字典
- 8. 数据库编程
 - 8.1. 标准SQL缺点和SQL编程技术优点
 - 8.2. 存储过程
 - 8.3. JDBC编程
- 10. 数据恢复技术
 - 10.1. 事务的概念及事务的4个特性及恢复技术能保证事务的哪些特性
 - 10.2. 故障种类(数据库的高可用性技术)
 - 10.3. 恢复技术
 - 10.4. 为什么事务的非正常结束会影响数据库数据的正确性?
 - 10.5. 登记日志文件时为什么必须先写日志文件，后写数据库?
 - 10.6. 针对不同的故障的恢复策略和方法
 - 10.6.1. 事务内部故障
 - 10.6.2. 系统故障
 - 10.6.3. 介质故障
 - 10.7. 什么是检查点记录，包括什么内容
 - 10.8. 具有检查点恢复技术有什么优点?
 - 10.9. 使用检查点方法进行恢复的步骤
 - 10.10. 什么是数据库镜像？它有什么用途
 - 10.11. 为什么UNDO是反向扫描日志，REDO是正向扫描日志
 - 10.12. 恢复系统是否可以保证事务的原子性和持续性
- 11. 并发控制
 - 11.1. 数据库中为什么要并发控制？并发控制技术能保证事务的哪些特性？
 - 11.2. 并发操作会产生哪几类数据的不一致？用什么方法可以避免
 - 11.3. 什么是封锁？封锁类型有哪几种？
 - 11.4. 三级协议分别能解决哪些问题
 - 11.5. 什么是活锁，产生原因和解决办法
 - 11.6. 什么是死锁，解决死锁的办法
 - 11.7. 什么样的并发调度是正确的调度
 - 11.8. 如何保证并发调度的正确性
- 12. NoSQL和New数据库

1. 数据库系统基础

1.1 数据库系统概述

1.1.1 基本概念

数据：描述事物的符号记录

数据库：数据库是长期储存在计算机内、有组织的、可共享的大量数据的集合

DBMS：数据库管理系统

数据库系统：数据库系统是由数据库、数据库管理系统（及其应用开发工具）、应用程序和数据库管理员(DataBase Administrator,DBA)组成的存储、管理数据的系统

DBMS的体系结构：3层

数据源(DATA SOURCE)：远程数据库的别名

ODBC：开发工具访问数据库统一的平台

JDBC：Java数据库连接



乔卿

已关注

481



3677

13

分享

打赏



专栏

1.1.2 数据独立性

数据独立性包括物理独立性和逻辑独立性（本质：希望不管怎么变，想不修改应用程序）

物理独立性：指用户的应用程序与数据库中数据的物理存储是相互独立的

逻辑独立性：指用户的应用程序与数据库的逻辑结构是相互独立的

1.2 概念模型

实体(entity)：客观存在并可相互区别的事物，虚拟的概念

属性(attribute)：实体所具有的某一特性，共同的性质

码(key)，键：唯一标识实体的属性集

实体类型(entity type, class)：用实体名及其属性名集合来抽象和刻画同类实体。例如学生(学号, 姓名, 性别, 出生年月日, 所在院系, 入学时间)就是

实体集(entity set)：同一类型实体的集合。例如，全体学生就是一个实体集

联系(relationship)：实体集内的单个实体之间的联系通常是指不同实体集之间的联系，实体之间的联系有一对一、一对多和多对多等多种类型

Association（联系、关联）

学生登记表					
学号	姓名	年龄	性别	系名	年级
2013004	王小明	19	女	社会学	2013
2013006	黄大鹏	20	男	商品学	2013
2013008	张文斌	18	女	法律	2013
...	...	...	...	...	...

图 1.14 关系模型的数据结构

关系数据库 系统采用关系模型作为数据的组织方式

基本概念：

关系(ralation)：一个关系对应通常说的**一张表**

元组(tuple)：**表中的一行**即为一个元组

属性(attribute)：表中的**一列**即为一个属性，给每一个属性起一个名称即为属性名

码,键(key)：表中的某个属性组，它可以**唯一确定一个元组**

域(domain)：域是一组具有相同数据类型的值的集合。即某个**属性的取值范围**

分量：元组中的一个属性值

关系模式：对关系的描述，一般表示为

关系名(属性1, 属性2, ..., 属性n)

1.3 数据库系统的结构



乔卿 已关注

481



3677

13

分享

打赏



专栏

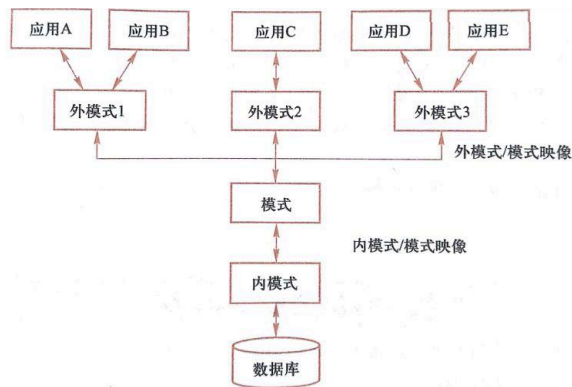


图 1.16 数据库系统的三级模式结构

1.模式(schema)

模式也称逻辑模式，是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图，不同的dbms表示模式的方法不一样！

Create schema schema_name ;

Schema : object (user, table , view , function, procedure,index,...);

Schema = user(MYSQL , ORACLE,...)

Schema : object (table , view , function, procedure,index,...);

Create user user_name identified by pass_wd;

2.外模式(external schema)

它是数据库用户能够看见和使用的局部数据的逻辑结构和特征的描述(

(你有权限可我也看到的东西！ VIEW)

3.内模式(internal schema)

一个数据库只有一个内模式。它是数据物理结构和存储方式的描述

2. 关系数据库

2.1 关系数据结构及形式化定义

基本概念：

候选码（key）：某一属性组的值能唯一地标识一个元组，而其自己不能，则称该属性组为候选码(candidate key)

主码：若一个关系有多个候选码，则选定其中一个为主码(primary key)，主码只可以有一个

主属性(prime attribute)：候选码的诸属性称为（主属性）

非主属性：不包含在任何候选码中的属性

全码(all key)：关系模式的所有属性是这个关系模式的候选码；

代理键（id，序列号、序号，系统自动生成，自增长）

2.2 关系的完整性

2.2.1 实体完整性

若属性A是基本关系的主属性，则A不能取空值

2.2.2 参照(引用)完整性

主（父亲）、从（子）



乔卿

已关注

481



3677

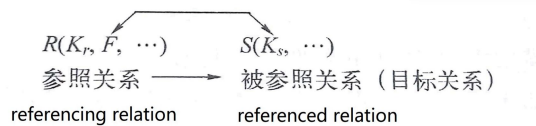
13

分享

打赏



专栏



2.3 关系代数

1.选择(selection)

选择是在关系R中选择满足给定条件的诸元组，记作

$$\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{真}\}$$

其中F表示选择条件，它是一个逻辑表达式

例如：查询信息系（IS系）的学生

$$\sigma_{\text{Sdept}='IS'}(\text{Student})$$

2.投影(projection)

关系R上的投影是从R中选择出若干属性列组成新的关系，记作

$$\pi_A(R) = \{t[A] | t \in R\}$$

其中A为R中的属性列

例如：查询学生的姓名和所在系

$$\pi_{\text{Sname}, \text{Sdept}}(\text{Student})$$

3.连接(笛卡尔积)

$$R \bowtie_{A \theta B} S = \{t_r t_s | t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B]\}$$

其中，A和B分别为R和S上列数相等且可比的属性组， θ 是比较运算符

- 等值连接： θ 为“=”的连接运算
- 自然连接：是一种特殊的等值连接。它要求两个关系中进行比较的分量必须是同名的属性组，并且在结果中把重复的属性列去掉 ????
- 悬浮元组(dangling tuple)：因为公共属性组不相等被舍弃的元组
- 外连接(outer join)：把悬浮元组保存在结果关系中
- 左外连接(left outer join或left join)：只保留左边关系R中的悬浮元组
- 右外连接(right outer join或right join)：只保留右边关系S中的悬浮元组

4.并UNION

5.差minus (except)

6.重命名 as

3. 关系数据标准语言SQL

删除 (drop、delete)

• Drop：删除数据库对象 (user table, view, index, fi

• Delete：删除数据



乔卿

已关注

481



3677

13

分享

打赏



专栏

修改 (alter , update)

- Alter 改数据库对象 (modify) (user table,view,index,function,procedure)
- Update 改数据!

3.1 数据定义

3.1.1 模式

1.定义模式

```
CREATE SCHEMA<模式名> AUTHORIZATION<用户名>
```

```
Create user <用户名> identified by <用户密码>;
```

2.删除模式

```
DROP SCHEMA<模式名><CASCADE|RESTRICT>
```

```
Drop user <用户名>;
```

```
Drop user <用户名> cascade;
```

3.1.2 表

1.定义基本表

```
CREATE TABLE<表名>(<列名><数据类型>[列级完整性约束条件]
    [,<列名><数据类型>[列级完整性约束条件]]
    ...
    [,<表级完整性约束条件>]);
```

例如:

[例 3.7] 建立学生选课表 SC。

```
CREATE TABLE SC
    (Sno CHAR(9),
    Cno CHAR(4),
    Grade SMALLINT,
    PRIMARY KEY (Sno,Cno), /*主码由两个属性构成，必须作为表级完整性进行定义*/
    FOREIGN KEY (Sno) REFERENCES Student(Sno),
    /*表级完整性约束条件，Sno 是外码，被参照表是 Student */
    FOREIGN KEY (Cno) REFERENCES Course(Cno)
    /*表级完整性约束条件，Cno 是外码，被参照表是 Course*/
);
```

2.修改基本表

```
ALTER TBAL<表名>
```

```
[ADD [COLUMN]<新列名><数据类型>[完整性约束]]
```

```
[ADD<表级完整性约束>]
```

```
[DROP[COLUMN]<列名>[CASCADE|RESTRICT]]
```

```
[DROP CONSTRAINT<完整性约束名>[RESTRICT|CASCADE]]
```

```
[ALTER COLUMN<列名><数据类型>];
```

3.删除基本表

```
DROP TABLE <表名> [RESTRICT|CASCADE]
```

3.1.3 索引

1.建立索引



乔卿

已关注

481



3677

13

分享

打赏



专栏

```
CREATE [UNIQUE][CLUSTER] INDEX <索引名>
```

```
ON <表名>(<列名>[<次序>][,<列名>[<次序>]]...);
```

2.修改索引

```
ALTER INDEX <旧索引名> RENAME TO <新索引名>;
```

3.2 数据查询

```
(SELECT [ALL|DISTINCT] <目标列表达式> [,<目标列表达式>]...
```

```
FROM <表名或视图名> [,<表名或视图名>...]((<SELECT 语句>) [AS] <别名>
```

```
[WHERE <条件表达式>]
```

```
[GROUP BY <列名1>
```

```
[HAVING <条件表达式>]]
```

```
[ORDER BY <列名2> [ASC|DESC]])
```

```
;
```

3.3 数据更新

3.3.1 插入元组

```
INSERT INTO <表名> [(<属性列1>[,<属性列2>...]) VALUES(<常量1>[,<常量2>]...)
```

3.3.2 插入子查询结果

```
INSERT INTO <表名>(<属性列1>[,<属性列2>...]) 子查询;
```

3.4 修改数据

```
UPDATE <表名>
```

```
SET <列名> = <表达式>[,<列名>=<表达式>]...
```

```
[WHERE<条件>;
```

3.5 删除数据

```
DELETE FROM <表名>
```

```
[WHERE<条件>;
```

3.6 视图

视图能够简化用户的操作（简化操作）

视图使用户能以多种角度看待同一数据（多角度）

视图对重构数据库提供了一定程度的逻辑独立性（逻辑独立性）

视图能对机密数据提供安全保护（安全保护）

适当利用视图可以更清晰地表达查询（清晰表达）

3.6.1 定义视图

```
CREATE VIEW <视图名>[(<列名>[,<列名>]...)]
```



乔卿

已关注

481



3677

13

分享

打赏



专栏

AS <子查询>

[WITH CHECK OPTION]

当更新、插入或删除时，WITH CHECK OPTION可以将子查询中的条件自动加上

3.6.2 删除视图

DROP VIEW <视图名> [CASCADE]

3.7 索引

Create index <索引名> on <表名> (<列名>[,<列名>]…)

Drop index <索引名>;

3.8 过程与函数

3.8.1 过程

Create procedure <过程名> ([(<参数 in 参数类型>[,<参数 in out参数类型>]…)

As

Begin

...

End;

3.8.2 函数

Create function <函数名> ([(<参数 in 参数类型>[,<参数 in参数类型>]…)

Return <结果的数据类型>

Begin

...

Return <结果表达式>;

End;

4.1 数据库安全性控制

保护数据库以防止不合法使用所造成的的数据泄露、更改或破坏

4.2 对数据库不安全的因素

非授权用户对数据库的恶意存取和破坏

数据库中重要或敏感的数据被泄露

安全环境的脆弱性

4.3 数据库安全性控制

用户身份鉴别

存取控制

自主存取控制，用户对不同数据库队友有不同的存取权限

强制存取控制，每一个数据库对象被标以一定的密级，只有具有合法许可证的用户才能存取

视图机制

审计



乔卿

已关注

481



3677

13

分享

打赏



专栏

4.4. 审计

审计功能是用户对数据库执行操作时，将这些操作自动记录到数据库的审计日志中

因为数据库安全保护措施不是完美无缺的。利用数据库审计信息，数据库管理人员可以重现并分析导致数据库现有状态的一系列事件，找出非法存取数据内容等。

4.1.1 授权：授予与收回

1.GRANT

GRANT <权限>[,权限]...

ON <对象类型><对象名>[,<对象类型><对象名>]...

TO <用户>[,<用户>]...

[WITH GRANT OPTION]

如果指定了WITH GRANT OPTION子句，则获得某种权限的用户还可以把这种权限再授予其他的用户

2.REVOKE

REVOKE<权限>[,<权限>]...

ON <对象类型><对象名>[,<对象类型><对象名>]...

FROM <用户>[,<用户>]...[CASCADE|RESTRICT]

4.1.2 数据库角色（按岗位职责，组织权限）

1.角色的创建

CREATE ROLE <角色名>

2.给角色授权

GRANT <权限>[,<权限>]...

ON <对象类型>对象名

TO <角色>[,<角色>]...

3.将一个角色授予其他的角色和用户

GRANT <角色1>[,<角色2>]...

TO <角色3>[,<用户1>]...

[WITH ADMIN OPTION]

如果指定了WITH ADMIN OPTION子句，则获得某种权限的角色或用户还可以把这种权限再授予其他的角色

4.角色权限的收回

REVOKE <权限>[,<权限>]...

ON <对象类型><对象名>

FROM <角色>[,<角色>]...

6. 关系数据理论

6.1 规范化



乔卿

已关注

481



3677

13

分享

打赏



专栏

一个低一级范式的关系模式通过模式分解(schema decomposition)可以转换为若干个高一级范式的关系模式的集合，这种过程就叫规范化(normalization)

6.1.1 函数依赖

属性闭包！

阿姆斯特朗公理！

等价的最少的函数依赖集！

6.1.2 第二范式(2NF)

若 $R \in 1NF$ ，且每一个非主属性完全函数依赖于任何一个候选码，则 $R \in 2NF$

例如：SLC的码为(Sno,Cno)，Sdept和Sloc依赖于Sno，不依赖于Cno，则不满足第二范式

6.1.3 第三范式(3NF)

若 $R \in 2NF$ ，且非主属性不能有传递依赖

例如：SLC的码为(Sno,Cno)，Sno决定Sdept，Sdept决定Sloc，存在 $Sno \rightarrow Sdept \rightarrow Sloc$ 这样的传递依赖，则不满足第三范式

6.1.4 BCNF(Boyce Codd Normal Form)

若 $R \in 3NF$ ，且不能有主属性对码的部分依赖和传递依赖

7. 数据库设计

7.1. 数据库设计概述

数据库设计是指对于一个给定的应用环境，构造（设计）优化的数据库逻辑模式和物理结构，并据此建立数据库及其应用系统，使之能够有效的存储和管理各种用户的应用需求，包括信息管理要求和数据操作要求。

7.2. 数据库设计的基本步骤

需求分析（建立数据字典）

概念结构设计(E-R图建立，应用需求抽象为信息世界的结构)

逻辑结构设计(E-R图转成相应的数据模型，在基本表基础上构建外模式)

物理结构设计(选择物理存储安排，建立索引，构建内模式)

数据库实施

数据库运行和维护

需求分析和概念设计独立于任何数据库管理系统

逻辑设计和物理设计与选用的数据库管理系统密切相关

7.3. 数据字典

它是关于数据库中数据的描述，即元数据，而不是数据本身。数据字典是在需求分析阶段建立，在数据库设计过程中不断修改、充实、完善的。

通常包括：数据项、数据结构、数据流、数据存储和处理。

数据库的概念模型向物理模型的转换规则！

8. 数据库编程

8.1. 标准SQL缺点和SQL编程技术优点

标准SQL是非过程化的查询语言，缺少流程控制能力，难以实现应用业务的逻辑控制

SQL编程技术可以有效克服SQL语言实现复杂应用方面的不足，提高应用系统和数据库管理系统间的互操作性。



乔卿

已关注

481



3677

13

分享

打赏



专栏

8.2. 存储过程

存储过程是由过程化SQL语句书写的过程，这个过程经编译和优化后存储在数据库服务器中。

运行效率高，因为是被编译后保存在数据库中的

降低了客户机和服务器的通信量

方便实施企业规则

8.3. JDBC编程

是为解决异构数据库间的数据共享而产生的，它建立了一组规范，并提供了一组访问数据库的应用程序编程接口（API）。JDBC具有两重约束力：一方面开发，另一方面规范关系数据库管理系统应用接口

10. 数据恢复技术

10.1. 事务的概念及事务的4个特性及恢复技术能保证事务的哪些特性

ACID

原子性，事务是数据库中的逻辑工作单元，里面的操作要么全做，要么不做

一致性，事务执行结果必须是一个事务从一个一致性状态转到另一个一致性状态

隔离性，一个事务的执行不能受其他事务的影响

持久性，一旦一个事务完成提交了，那么对数据库的改变是永久的，保证了事务的原子性和隔离性

10.2. 故障种类(数据库的高可用性技术

（备份、恢复！复制、分区、主库（master）、从库（standby），cluster，data guard）

事务内部的故障

系统故障

介质故障

计算机病毒

10.3. 恢复技术

恢复机制两个关键问题：如何建立冗余数据，以及如何利用这些冗余数据实施数据库恢复。建立数据库冗余数据最常用的技术是数据转储和登记日志文件

10.4. 为什么事务的非正常结束会影响数据库数据的正确性？

如果数据库运行过程中出现故障导致事务非正常结束，有些事务尚未完成就被中断，那么就破坏了事务的原子性，这些事务的一部分操作已经写入了数据库处于不一致的状态，例如银行转账。

10.5. 登记日志文件时为什么必须先写日志文件，后写数据库？

因为这是两个不同的操作，如果中间发送故障，两个只能完成一个。如果先写了数据库操作，而在日志中没有这个记录，那么就无法恢复这个修改了。如志，但没有写入数据库。那么恢复时只要多执行一次UNDO操作，并不会影响数据库操作。

10.6. 针对不同的故障的恢复策略和方法

10.6.1. 事务内部故障

反向扫描文件日志，查找到该事务的更新操作

对该事务的更新操作执行逆操作，直到读到改事务的开始标记

10.6.2. 系统故障

正向扫描日志文件，找出在故障前已经提交的事务段



乔卿

已关注

481



3677

13

分享

打赏



专栏

对未完成队列执行中各个事务执行UNDO操作

对已经提交的各个事务执行REDO操作

10.6.3. 介质故障

装入最新的数据库后备副本，使数据库恢复到最近一次转储时的一致性状态

装入转储结束时刻的日志文件副本

启动系统恢复命令，由DBMS完成恢复功能

10.7. 什么是检查点记录，包括什么内容

检查点记录是一类新的日志记录。它的内容包括：

- 建立检查点时刻所有正在执行的事务清单
- 这些事务的最近一个日志记录的地址

10.8. 具有检查点恢复技术有什么优点？

节约时间，利用日志技术进行数据库恢复时，恢复子系统必须搜索整个日志，这将耗费大量时间

需要REDO的操作实际上已经将它们的更新操作结果写到数据库了，恢复子系统又重新执行了这些操作，浪费了大量时间

10.9. 使用检查点方法进行恢复的步骤

找到最后一个检查点记录在日志文件的地址，由该地址在日志文件中找到最后一个检查点记录

由该检查点记录得到检查点建立时刻所有正在执行的事务清单，建立两个事务队列UNDO和REDO

从检查点开始正向扫描日志文件，根据提交与否选择重做还是撤销

对队列执行事务操作

10.10. 什么是数据库镜像？它有什么用途

自动将整个数据库或者关键数据复制到另一个磁盘上。每当主数据库更新时，把更新后的数据复制过去，即自动保证镜像数据和主数据的一致性。

用途：

- 数据恢复
- 提升数据库利用率。例如被加了排他锁，可以在镜像数据库上读。

10.11. 为什么UNDO是反向扫描日志，REDO是正向扫描日志

UNDO是恢复到第一个失败的事务就OK了，正向做不到。REDO是恢复到最后一个成功的事务之后。

10.12. 恢复系统是否可以保证事务的原子性和持续性

UNDO保证原子性，REDO保证持续性

11. 并发控制

11.1. 数据库中为什么要并发控制？并发控制技术能保证事务的哪些特性？

数据库时共享资源，通常有多个事务同时执行。当多个事务同时并发地存取时就会产生同时读/写同一个数据。若对并发操作不加以控制就可能导致存取破坏事务的一致性。

并发控制保证了事务的一致性和隔离性

11.2. 并发操作会产生哪几类数据的不一致？用什么方法可以避免

丢失修改，两个事务对同一个数据同时进行修改，那

不可重复读，一个事务读了某一数据以后，另一个事



乔卿

已关注

👍 481



🌟 3677

💬 13

🔗 分享

💰 打赏



专栏

读取脏数据，一个事务修改了某一数据并把其写回磁盘，另一个事务读取了这个数据以后，之前那个事务因为某种原因撤销了。

避免不一致性的方法就是并发控制，常用的并发控制有封锁法、时间戳法、乐观控制法、多版本并发控制法等

11.3. 什么是封锁？封锁类型有哪几种？

封锁就是事务T对某一数据进行操作前，先向系统发送请求，对其加锁，加锁以后事务就对这个数据有了一定的控制权，在事务T释放锁之前，其他事务不能行更新或者读取

封锁类型有：排它锁、共享锁

11.4. 三级协议分别能解决哪些问题

加X锁，直到事务结束再释放。解决了丢失修改问题。

在1基础上加S锁，读完后可以释放，解决了读取脏数据的问题。

在1基础上加S锁，直到事务结束再释放，解决了不可重复读问题

11.5. 什么是活锁，产生原因和解决办法

当一系列封锁操作无法按照其正确顺序执行时，就可能导致事务无限等待某个封锁。

避免活锁的方法就是使用FCFS

11.6. 什么是死锁，解决死锁的办法

举例说明什么是死锁。

防止死锁的方法有两种：预防死锁、死锁诊断与解除

预防死锁有两种办法：一次封锁法、顺序封锁法

死锁诊断与解决：超时法、事务等待图法

解除法：选择处理死锁代价最小的事务，将其解除

```
Alter session kill 'sessionid,pid ';
```

11.7. 什么样的并发调度是正确的调度

可串行化的并发调度是正确的调度。可串行化调度定义：多个事务并发执行是正确的，当且仅当其结果与按某一次序执行的串行执行的结果相同。

11.8. 如何保证并发调度的正确性

冲突可串行化，使用两段锁协议

12. NoSQL和New数据库

自1970年以来，数据库技术发展到了第四代，第一代是60~70年早期的层次和网状数据库系统，第二代是70年代中期开始的关系数据库系统，第三代是80年代后期出现的面向对象数据库系统和以及90年出现的对关系模型进行扩展，使关系数据库系统具有面向对象功能。

自2000年以来，Web技术、社交网络、移动计算和物联网的进步导致全球范围应用程序生成的半结构化、非结构化数据呈现爆炸式增长态势，此类应用有不同的需求，包括水平可扩展能力、支撑大规模海量数据处理能力，系统要具有 **高可用性** 和容错能力以应对硬件和软件故障，并且可以通过利用额外资源来请求，从而提高查询处理速率，这就导致了第四代数据库技术NoSQL和NewSQL技术的出现。

NoSQL数据库的关键特征是：(1)采用图形、树、键值、文档等表格模型以外的数据模型；(2)数据库设计时没有固定的模式定义；(3)提供对水平可扩展性；(4)具有最终一致性而不是强一致性。NoSQL的另一个重要概念是CAP定理,它对 **分布式系统** 至关重要的属性提供了一些约束，即：一致性、可用性和分区容错性。CAP定理表明，分布式系统在任何给定时间最多只能满足其中2个属性。

Json格式！！

NewSQL是最新一代的数据库系统，它不仅提供NoSQL数据库系统具有的可扩展性和可用性，还保留了传统关系数据库的ACID特性、关系数据模型和SQL

数据库sql期末总复习

数据库总复习，内有众多章节的详细ppt，希望能给大家



乔卿

已关注

481



3677

13

分享

打赏



专栏